

Esercizio 2 (9 punti) Data una stringa $X = x_1, x_2, \dots, x_n$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i + 1, j - 1) & \text{se } (i < j - 1) \text{ e } (x_i = x_j) \\ \sum_{k=i}^{j-1} (\ell(i, k) + \ell(k + 1, j)) & \text{se } (i < j - 1) \text{ e } (x_i \neq x_j). \end{cases}$$

1. Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{REC_L}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$. (vedo (1,n) e so che la scansione è per diagonale)
2. Si determini la complessità *al caso migliore* $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Domanda A (6 punti) Si determini la soluzione asintotica della seguente equazione di ricorrenza:

$$T(n) = 4T(n/3) + 2n^2 + 1$$

Caso 3 del MT e si risolve con condizione di regolarità.

Esercizio 1 (10 punti) Un array di interi $A[1..n]$ si dice *triangolare* se esiste $q \in [1, n]$ tale che $A[1..q]$ è ordinato in modo crescente e $A[q..n]$ è ordinato in modo decrescente. Realizzare una funzione $\text{maxTr}(A)$ che dato un array triangolare $A[1..n]$, senza elementi ripetuti, ne trova il massimo. Valutarne la complessità.

Domanda 28 Sia T un albero binario i cui nodi x hanno i campi $x.\text{left}$, $x.\text{right}$, $x.\text{key}$. L'albero si dice un *sum-heap* se per ogni nodo x , la chiave di x è maggiore o uguale sia alla somma delle chiavi nel sottoalbero sinistro che alla somma delle chiavi nel sottoalbero destro.

Scrivere una funzione $\text{IsSumHeap}(T)$ che dato in input un albero T verifica se T è un sum-heap e ritorna un corrispondente valore booleano. Valutarne la complessità.

Esercizio 1 (9 punti) Realizzare una funzione $\text{append}(T, x, z)$ che dato un albero binario di ricerca T , un suo nodo *foglia* x ed un nuovo nodo z , verifica se è possibile inserire z come figlio (destro o sinistro) di x in T . In caso affermativo ritorna l'albero esteso, altrimenti ritorna **nil**. (Assumere che i nodi dell'albero abbiano gli usuali campi $x.\text{left}$, $x.\text{right}$, $x.p$, $x.\text{key}$ e la radice sia $T.\text{root}$.) Valutarne la complessità.

Esercizio 2 (10 punti) Dato un insieme di n numeri reali positivi e distinti $S = \{a_1, a_2, \dots, a_n\}$, con $0 < a_i < a_j < 1$ per $1 \leq i < j \leq n$, un $(2,1)$ -boxing di S è una partizione $P = \{S_1, S_2, \dots, S_k\}$ di S in k

sottoinsiemi (cioè, $\bigcup_{j=1}^k S_j = S$ e $S_r \cap S_t = \emptyset, 1 \leq r \neq t \leq k$) che soddisfa inoltre i seguenti vincoli:

$$|S_j| \leq 2 \quad \text{e} \quad \sum_{a \in S_j} a \leq 1, \quad 1 \leq j \leq k.$$

In altre parole, ogni sottoinsieme contiene al più due valori la cui somma è al più uno. Dato S , si vuole determinare un $(2,1)$ -boxing che minimizza il numero di sottoinsiemi della partizione.

1. Scrivere il codice di un algoritmo greedy che restituisce un $(2,1)$ -boxing ottimo in tempo lineare. (Suggerimento: si creino i sottoinsiemi in modo opportuno basandosi sulla sequenza ordinata.)
2. Si enunci la proprietà di scelta greedy per l'algoritmo sviluppato al punto precedente e la si dimostri, cioè si dimostri che esiste sempre una soluzione ottima che contiene la scelta greedy.